

PriorityQueueList.h

```
#ifndef _PRIORITYQUEUELIST_H
#define _PRIORITYQUEUELIST_H

#include <stdbool.h>
#include <stdlib.h>

#define boolean bool
#define Nil NULL

/* Deklarasi type elemen */
typedef int ElType;

/* Priority Queue dengan representasi berkaitan dengan pointer */
typedef struct tNode *Address;
typedef struct tNode
{
    ElType info;
    int priority;
    Address next;
} Node;

/* Type PriorityQueue dengan HEAD */
typedef struct
{
    Address addrHead; /* alamat elemen pertama */
} PriorityQueue;

/* Selektor */
#define NEXT(p) (p)->next
#define INFO(p) (p)->info
#define PRIORITY(p) (p)->priority
#define ADDR_HEAD(q) (q).addrHead
#define HEAD(q) (q).addrHead->info

/* Prototype manajemen memori */
Address newNode(ElType x, int pr);
/* Mengembalikan alamat sebuah Node hasil alokasi dengan info = x dan priority =
pr,
atau NIL jika alokasi gagal */

void delNode(Address P);
/* I.S. P adalah hasil alokasi, P != Nil */
/* F.S. Alamat P didealokasi, dikembalikan ke sistem */

boolean isEmpty(PriorityQueue q);
/* Mengirim true jika q kosong: ADDR_HEAD(q)=Nil and ADDR_TAIL(q)=Nil */

int length(PriorityQueue q);
/* Mengirimkan banyaknya elemen queue. Mengirimkan 0 jika q kosong */
```

```
/** Kreator */
void CreateQueue(PriorityQueue *q);
/* I.S. sembarang */
/* F.S. Sebuah q kosong terbentuk */
/** Primitif Enqueue/Dequeue */

void enqueue(PriorityQueue *q, ElType x, int pr);
/* Proses: Mengalokasi x dengan priority pr dan menambahkan x pada q
jika alokasi berhasil; jika alokasi gagal q tetap */
/* I.S. q mungkin kosong */
/* F.S. x menjadi elemen q sesuai dengan urutan prioritas */

void dequeue(PriorityQueue *q, ElType *x, int *pr);
/* Proses: Menghapus x pada bagian HEAD dari q dan mendealokasi elemen HEAD */
/* Pada dasarnya operasi deleteFirst */
/* I.S. q tidak mungkin kosong */
/* F.S. x = nilai elemen HEAD pd I.S., HEAD "mundur" */

#endif /* _PRIORITYQUEUELIST_H */
```

PriorityQueueList.c

```
#include "priorityQueueList.h"

Address newNode(ElType x, int pr)
{
    Address p = (Address)malloc(sizeof(Node));
    if (p != Nil)
    {
        INFO(p) = x;
        PRIORITY(p) = pr;
        NEXT(p) = Nil;
    }
    return p;
}

void delNode(Address P)
{
    if (P != Nil)
    {
        free(P);
    }
}

bool isEmpty(PriorityQueue q)
{
    return (ADDR_HEAD(q) == Nil);
}

int length(PriorityQueue q)
{
    Address p = ADDR_HEAD(q);
    int count = 0;
    while (p != Nil)
    {
        count++;
        p = NEXT(p);
    }
    return count;
}

void CreateQueue(PriorityQueue *q)
{
    ADDR_HEAD(*q) = Nil;
}
```

```

void enqueue(PriorityQueue *q, ElType x, int pr)
{
    Address p = newNode(x, pr);
    if (p != Nil)
    {
        if (isEmpty(*q) || PRIORITY(p) < PRIORITY(ADDR_HEAD(*q)))
        {
            NEXT(p) = ADDR_HEAD(*q);
            ADDR_HEAD(*q) = p;
        }
        else
        {
            Address curr = ADDR_HEAD(*q), prev = Nil;
            while (curr != Nil && PRIORITY(curr) <= PRIORITY(p))
            {
                prev = curr;
                curr = NEXT(curr);
            }
            NEXT(p) = curr;
            NEXT(prev) = p;
        }
    }
}

void dequeue(PriorityQueue *q, ElType *x, int *pr)
{
    Address del = ADDR_HEAD(*q);
    (*x) = INFO(del);
    (*pr) = PRIORITY(del);
    ADDR_HEAD(*q) = NEXT(del);
    delNode(del);
}

```